

Code-Layout, Readability and Reusability

Date	14 July 2026
Team ID	
Project Name	Personalized Networking Assistant
Maximum Marks	5 Marks

Code-Layout, Readability and Reusability:

This document evaluates the quality of the **Personalized Networking Assistant** codebase in terms of organization, readability, and reusability. The project follows a modular full-stack TypeScript architecture using React, Node.js, Express, SQLite, Google Gemini, and Wikipedia-based factual verification.

Code Layout Checklist:

S.No	Code Quality Parameter	Description	Followed (Yes / No / Partial)	Remarks
1	Consistent Indentation	Uniform formatting and indentation are used throughout the TypeScript codebase.	Yes	Consistent structure across client and server files.
2	Proper File Structure	UI components, shared types, routes, and backend services are logically separated.	Yes	React frontend and Express services are clearly organized.
3	Meaningful Variable Names	Variables and shared types clearly reflect their purpose.	Yes	Typed profile, API, and option structures improve clarity.
4	Function / Method Names	Functions and service methods use descriptive names.	Yes	Service responsibilities are easy to identify.
5	Code Comments	Important logic is supported by clear structure and targeted documentation.	Partial	Additional inline comments can be added where needed.
6	Modular Design	Code is divided into reusable UI components and independent backend services.	Yes	Gemini, fact-checking, database, and user services are separated.
7	No Redundant Code	Reusable components and helper services reduce duplicated logic.	Yes	Shared components and service functions are reused.
8	Error Handling	Validation and controlled service boundaries improve reliability.	Yes	External integrations are handled through dedicated services.

Reusable Components / Modules:

S.No	Component / Module Name	Language / Technology	Where Reused	Reusability Level (High / Medium / Low)
1	Reusable UI Components	React, TypeScript	Shared interface elements across the application.	High
2	Logo Component	React, TypeScript	Reusable branded interface component.	High
3	Shared Types	TypeScript	Profiles, API contracts, enums, and options.	High
4	Gemini Service	TypeScript, Google Gemini	AI-powered personalized content generation.	High
5	Fact-Check Service	TypeScript, Wikipedia API	Factual lookup and verification workflow.	High
6	Database Service	TypeScript, SQLite	Profiles, history, sessions, ratings, and feedback.	High
7	User Service	TypeScript, bcryptjs	User account and profile operations.	High
8	Express Router	Node.js, Express, TypeScript	API endpoints and server routing.	High

Overall Code Quality Assessment:

Aspect	Rating (1-5)	Comments
Code Layout & Structure	5	Clear separation between UI, shared types, routing, and backend services.
Readability	5	Strong TypeScript typing and meaningful modular boundaries improve readability.
Reusability	5	Reusable React components and independent services support code reuse.
Documentation / Comments	4	The architecture is clear; targeted inline documentation can be expanded further.
Overall Score	4.75	The project demonstrates strong modularity, maintainability, and reusable full-stack design.